

From local commands to verified delivery.

A five-slide walkthrough of what I built, what I proved, and what I would improve.

PRESENTED BY

Mark Andrei Castillo

Local WSL walkthrough | checkpoint evidence

THE STORY

Linux

- > Docker
- > CI/CD
- > Kubernetes HA
- > Part 5 reflection

Rule: local action -> captured result -> checkpoint meaning

I built the foundation before automating delivery

SYSTEM_HEALTH.SH OUTPUT

[illegible]

LOCAL WSL CAPTURE

FINAL COMPOSE RUN

[illegible]

LOCAL WSL CAPTURE

RESULT

Linux scripts made state observable.

Compose proved API, UI,
and MySQL could run
together.

LOCAL ACTIONS

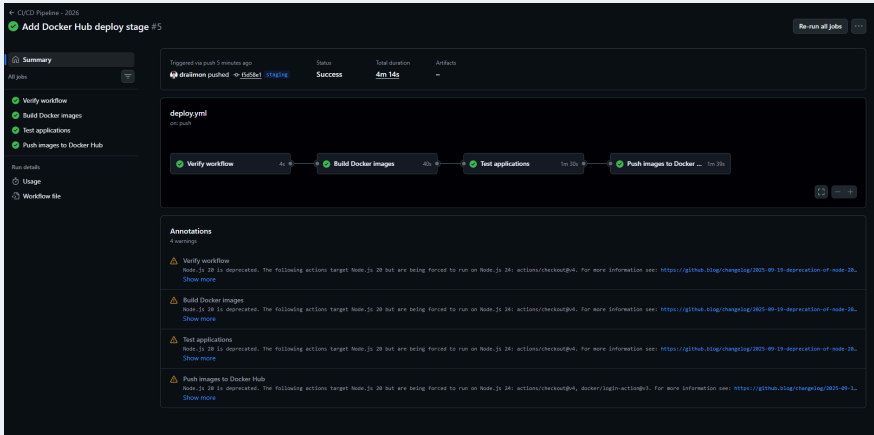
```
$ bash part1-linux/system_health.sh
$ bash part1-linux/backup.sh
$ bash part1-linux/log_analysis.sh
$ docker-compose ps
$ curl http://localhost:8000/
```

THOUGHT PROCESS

I checked the real source and fixed the first local failure before moving on. The API uses MySQL, so the database image and driver had to match the cloned application.

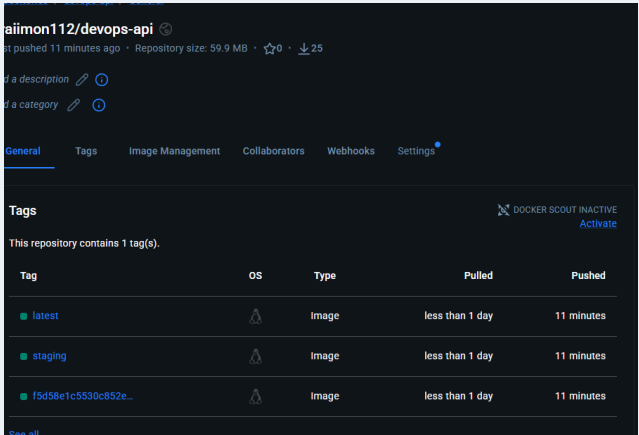
The pipeline turns a local push into a checked release path

SUCCESSFUL WORKFLOW GRAPH



CHECKPOINTED CI EVIDENCE

PUBLISHED IMAGE TAGS



CHECKPOINTED REGISTRY EVIDENCE

WORKFLOW ORDER

Verify -> Build -> Test -> Push -> Notify

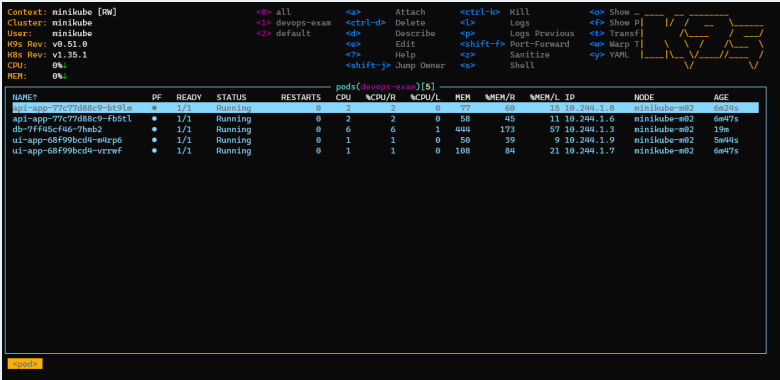
WHY THIS CONFIGURATION

Fresh jobs declare their own setup; tests run before publishing; SHA, staging, and latest tags preserve traceability.

PART 4 | HIGH AVAILABILITY

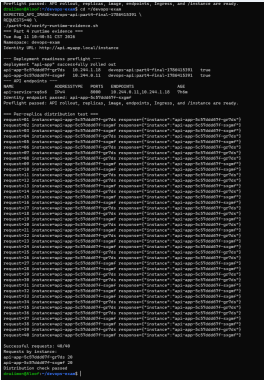
I tested the runtime, not only the Kubernetes YAML

PODS AND NODES READY



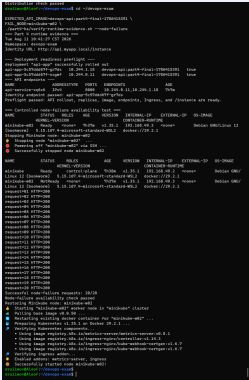
LOCAL WSL CAPTURE

40-REQUEST DISTRIBUTION



LOCAL WSL CAPTURE

NODE-FAILURE RESULT



LOCAL WSL CAPTURE

DESIGN

Two API replicas, Services, Ingress, probes, HPA, PDB, rolling updates, and topology spread across a two-node Minikube cluster.

PROOF

Preflight checked image consistency, EndpointSlice membership, routing, and /instance. Captured result: 20 responses per replica and 20/20 HTTP 200 during the node test.

The strongest decision was to make each boundary testable

CHALLENGES -> FIXES

Missing source -> cloned real apps

Database mismatch -> MySQL alignment

Fresh CI runners -> explicit job setup

Green rollout -> runtime preflight

TRADE-OFFS

Minikube is reproducible and exam-friendly, but not production control-plane HA.

Mutable staging is convenient; SHA tags and preflight improve traceability.

REPOSITORY WALKTHROUGH

README.md

part1-linux/

part2-docker/

.github/workflows/deploy.yml

part4-ha/k8s/

part4-ha/verify-runtime-evidence.sh

documentation/

With more time: required image scanning, immutable deployment digests, a tested rollback, and production observability.

01. Opening and presentation scope

SAY

Good day. I am Mark Andrei Castillo, and this is my Part 5 solution presentation for the Junior DevOps Engineer Exam 2026. I will present the work as a walkthrough: what I ran locally, what I captured, and what the checkpoint evidence allows me to claim. I will keep the story focused on five slides: foundation, delivery, high availability, and my engineering decisions.

SHOW / DO

WALKTHROUGH ACTION

Start with the title slide. Mention that the detailed script is included immediately after the five visual slides in this PDF.

EVIDENCE NOTE

The local execution boundary matters: the repository stores the implementation and evidence, while the Linux, CI, and Kubernetes work was performed and captured from the documented environments.

IF ASKED

The presentation requirements are approach, architecture, thought process, tools/configuration, challenges, resolutions, trade-offs, improvements, and a repository walkthrough. The five slides cover each of those.

02. Parts 1 and 2 foundation

SAY

I worked from the foundation upward. In Part 1, I practiced the Linux commands and turned repeated checks into Bash scripts: system health, backup, log analysis, and scripting. In Part 2, I checked that the actual FastAPI and Next.js source was in the build context, then built separate API and UI images with a MySQL Compose dependency. The important result was not only that images built; the API, UI, and database became healthy and the API responded to curl checks.

SHOW / DO

WALKTHROUGH ACTION

Show the two evidence screenshots. Point to `system_health.sh`, `backup.sh`, and `log_analysis.sh`, then show `docker-compose ps` and the API root check.

EVIDENCE NOTE

The first local Docker attempt exposed missing source and database/driver alignment issues. I corrected those before treating the stack as ready. This is why the deck distinguishes a build result from an application readiness result.

IF ASKED

I kept API and UI separate because they have different runtimes, dependencies, ports, and scaling boundaries. Compose provides a repeatable local feedback loop with MySQL.

03. Part 3 CI/CD

SAY

For Part 3, the local staging push handed work to GitHub Actions. The workflow order is Verify, Build, Test, Push, and Notify. Build validates the Compose configuration and images. Test installs dependencies, starts the real services, waits for readiness, and checks behavior. Only after those checks does the pipeline publish API and UI images to Docker Hub. SHA, staging, and latest tags make the output traceable.

SHOW / DO

WALKTHROUGH ACTION

```
Trace the successful workflow graph from left to right and then show the published registry tag evidence. Open  
.github/workflows/deploy.yml if asked.
```

EVIDENCE NOTE

The successful pipeline and registry publishing are captured evidence. Notification configuration is present, but I do not overclaim a new failure-notification capture or a live rollback test.

IF ASKED

Fresh GitHub-hosted jobs do not share hidden local state, so each job checks out the code and declares its own setup. That makes the pipeline more reproducible.

04. Part 4 high availability

SAY

For Part 4, I used a two-node Minikube cluster and raw Kubernetes manifests. The application has two API replicas and two UI replicas, stable Services, Ingress routing, readiness and liveness probes, HPA, a PodDisruptionBudget, rolling updates, and topology spread. I then used a runtime verifier instead of trusting YAML alone. The captured results show traffic reaching both replicas and 20 out of 20 successful HTTP responses while one node was NotReady.

SHOW / DO

WALKTHROUGH ACTION

Show the ready pods, the 40-request distribution, and the controlled node-failure screenshot. Point out that preflight checks image consistency, EndpointSlice membership, routing, and /instance.

EVIDENCE NOTE

This is the strongest HA boundary: the test exercises traffic and availability. The screenshots are local WSL captures; the Repl snapshot contains the verifier, manifests, documentation, and copied evidence.

IF ASKED

Minikube is appropriate for a reproducible exam environment. In production I would repeat the same workload design in managed Kubernetes with stronger control-plane and observability guarantees.

05. Part 5 decisions and closing

SAY

The main challenges were boundary problems: missing source in the first container build, database alignment, fresh CI runners, and a green rollout without traffic proof. I resolved them with source checks, MySQL alignment, explicit job setup, and runtime preflight. With more time, I would add image scanning, immutable digests, a tested rollback, and production observability. My main lesson is that reliable delivery is a chain of verified boundaries.

SHOW / DO

WALKTHROUGH ACTION

Walk through README.md, part1-linux/, part2-docker/, .github/workflows/deploy.yml, part4-ha/k8s/, verify-runtime-evidence.sh, and documentation/. End by inviting questions.

EVIDENCE NOTE

Keep completed evidence separate from configured behavior and future work. Do not present optional scanning or live rollback as already completed.

IF ASKED

The presentation is designed to be short, but every required Part 5 topic is available: approach, architecture, decisions, challenges, resolutions, trade-offs, improvements, repository walkthrough, Dockerfiles, CI/CD, and HA.